Edge cases (when the answer spans two chunks) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus the actual fix, not just a workaround). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

Chunking cuts a document into pieces on some fixed size, without knowing or caring where the sentences and facts actually are. Most of the time that's fine. But sometimes a single fact — a sentence that has to be read as one whole thing to make sense — gets cut right in the middle, with half of it landing in one chunk and the other half in the next chunk.

If your search only pulls back the top 1 chunk (`k=1`), and the fact you need is split across two chunks, you'll only ever get half the answer. Worse, it'll look like a normal, confident answer — just wrong or incomplete — and nothing will look broken.

This exercise asks you to make this happen on purpose: build one short document where you control exactly where the boundary falls, so a single important fact is split across two chunks. Then search for it and see what a small `k` actually returns.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

The exercise has three moving parts: fixed-size chunking, embedding + cosine-similarity search (the same search you built in `embedding_similarity`), and the `k` parameter that controls how many chunks a search returns.

Fixed-size chunking with plain slicing looks like this:

```
# chunking_practice.py — Edge cases section
for i in range(0, len(text), chunk_size):
    chunk = text[i:i + chunk_size]
```

This has no idea where a sentence ends — it just cuts every `chunk_size` characters. That's exactly the property you want here: pick a `chunk_size` deliberately small enough, relative to one long sentence, that the cut lands in the middle of the fact you care about.

The exact pieces:

- One long test sentence containing the whole fact.
- `chunk_by_chars(text, chunk_size)` sized so it produces exactly 2 chunks, with the cut in the middle of the fact.
- Reuse `get_embedding` and `cosine_similarity` unchanged.
- A `search(question, chunks, chunk_embeddings, k)` function, so you can compare `k=1` against `k=len(chunks)` directly.

A bigger `k` isn't actually a fix — it's a workaround. Even at `k=len(chunks)` (every chunk, guaranteed to include both halves), you only get lucky because this is a 2-chunk toy example. In a real knowledge base with thousands of chunks, a bigger `k` just makes it *more likely* the missing half scores well enough to make the cut — it still depends on that chunk actually ranking high enough, and a bigger `k` also means more irrelevant chunks get handed to the model as context (this document's Core Concepts calls this "lost in the middle").

The real fix is **chunking with overlap**: instead of cutting the document into non-overlapping blocks, each new chunk starts a little *before* where the previous one ended, so a fact sitting near a boundary ends up whole inside at least one chunk — not permanently split in two, regardless of `k`. The rule: any text up to `overlap` characters long always fits whole in some chunk. A fact longer than `chunk_size` can never fit, whatever the overlap — so check the fact's length first.

```
# chunking_practice.py — Edge cases section
def chunk_with_overlap(text: str, chunk_size: int, overlap: int) -> list[str]:
```

```
    chunks = []
    # smaller than chunk_size -- chunks now overlap
    step = chunk_size - overlap
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += step
    return chunks
```

Build the *same* boundary-splitting test document from Hint 2 below, chunk it with overlap instead of the plain non-overlapping slicer, and check: does the fact now appear whole inside at least one chunk, so even `k=1` can return the complete answer?

Difference between Basic and Intermediate: Basic demonstrates the problem and shows that a bigger `k` covers for it in this one small example. Intermediate also treats "just raise `k`" as the workaround it actually is, and builds the real fix — overlapping chunks — so the fact isn't split by the chunker in the first place, instead of hoping search recovers both halves after the fact.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
write one document: a single long sentence containing the whole fact
    (time, room, and who is expected to attend)

pick a small chunk_size so the sentence gets cut into exactly 2 chunks,
    with the cut landing in the middle of the fact

cut the document into chunks using the chunk_size, print each chunk
    so you can see exactly where the cut fell

get an embedding for each chunk

write a question that needs the whole fact to answer completely

get an embedding for the question

score every chunk against the question using cosine_similarity

search with k=1:
    print the top 1 chunk -> notice it only has half the answer

search with k=2 (all chunks):
    print the top 2 chunks -> notice together they have the full answer
```

Here is almost the whole thing — try running it and reading it line by line:

```
# chunking_practice.py — Edge cases section
import math
from openai import OpenAI

client = OpenAI()

document = (
    "The quarterly meeting will be held at 3 PM on Friday in Room 204, "
    "and everyone from the product and engineering teams is expected "
    "to attend without exception."
)
```

```
chunk_size = 80

chunks = []
for i in range(0, len(document), chunk_size):
    chunks.append(document[i:i + chunk_size])

for i, chunk in enumerate(chunks):
    print("chunk", i, ":", chunk)

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)
```

Expected output if you run just this: the 2 printed chunks — add embedding, scoring, and the `k=1` vs `k=2` comparison yourself, then check the Solution.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
# chunking_practice.py — Edge cases section
DOCUMENT: str = (
    "The quarterly meeting will be held at 3 PM on Friday in Room 204, "
    "and everyone from the product and engineering teams is expected "
    "to attend without exception."
)
CHUNK_SIZE: int = 80


def chunk_text(text: str, chunk_size: int) -> list[str]:
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i + chunk_size])
    return chunks
```

What's missing: a `search(question, chunks, chunk_embeddings, k)` function that scores and sorts, plus a `main()` that runs `k=1` and `k=len(chunks)` and compares. Write that yourself, then compare against the Solution.

Once that's working, add the real fix — overlap:

```
document = same quarterly-meeting sentence as before
the fact, "3 PM" ... "attend", is 101 characters long
rule: text up to `overlap` characters long always fits whole in a chunk
so: chunk_size = 140, overlap = 105   (80 is smaller than the fact itself)

chunk_with_overlap(document, chunk_size, overlap) -> should produce
    chunks where "3 PM" and "attend" sit inside the same chunk

embed each overlapping chunk

search with k=1 against the overlapping chunks:
    does the single top chunk now contain the whole fact,
    where the non-overlapping version at k=1 didn't?
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# chunking_practice.py — Edge cases section
def chunk_with_overlap(text: str, chunk_size: int, overlap: int) -> list[str]:
    chunks: list[str] = []
    step = chunk_size - overlap
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        # your turn: move i forward by `step`, not chunk_size
        ...
    return chunks


# your turn: build overlap_chunks from DOCUMENT using chunk_with_overlap,
# embed them, then run search(QUESTION, overlap_chunks, ..., k=1) and compare
# the result against the non-overlapping version's k=1 result
```

Finish `chunk_with_overlap` and the `k=1` comparison yourself, then compare all of your finished versions against the Solution.

Difference between Basic and Intermediate: Basic proves the failure exists and shows a bigger `k` covering for it in this small example. Intermediate removes the need for a bigger `k` at all — with overlap, `k=1` alone can return the complete fact, because the chunker never actually split it in the first place.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution